

텐서: 숫자를 담는 그릇

코드 해설 b01 ~ b35 · 2권 4장 4.1 ~ 4.11

오늘 볼 코드 35개

묶음	파일	핵심 질문
1. 텐서 만들기	b01 왜 텐서 · b02 목록 · b03 모양만 · b04 난수 · b24 생성기 · b05 NumPy	텐서는 왜 쓰고, 어떻게 만드나?
2. 모양과 자료형	b06 차원 · b07 사진 묶음 · b08 자료형 · b25 메모리 크기	shape와 dtype을 어떻게 읽나?
3. 인덱싱	b09 인덱싱 · b10 마스크 · b11 공유 · b26 where · b27 gather · b28 scatter	원하는 칸만 어떻게 꺼내고 바꾸나?
4. 모양 바꾸기	b12 reshape · b13 permute · b14 squeeze·cat·stack · b29 stride	숫자는 그대로, 모양만 어떻게 바꾸나?
5. 한꺼번에 계산	b15 원소별 · b16 브로드캐스팅 · b17 행렬 곱 · b18 축 접기 · b19 제자리 · b30 einsum · b31 정렬	반복문 없이 어떻게 계산하나?
6. 장치·저장·주의점	b20~b22 GPU · b32~b33 저장 · b34 NumPy와 차이 · b35 자주 만나는 오류	GPU로 옮기고, 저장하고, 오류를 읽으려면?
7. 실습	b23 5개 도시 × 12개월 기온 분석	배운 연산을 한 데이터에 모두 써 보기

코드 읽기 전에: 자주 나오는 파이썬 문법

문법	뜻	나오는 곳
`import time` / `import numpy as np`	도구 모음(모듈) 불러오기. as 뒤는 줄여 부를 이름	전체
`time.perf_counter()`	스톱워치: 지금 시각(초). 끝 시각 − 시작 시각 = 걸린 시간	b01, b22
`[식 for x in 목록]`	하나씩 꺼내 계산한 새 리스트 (리스트 컴프리헨션)	b01
`f"{x:,}"` `f"{x:.3f}"`	f-문자열: { } 안 값을 글자로. :, 천 단위 쉼표, :.3f 소수 셋째 자리	b01, b06, b25
`for a, b in [(…), (…)]`	짝으로 묶인 값을 두 변수에 나눠 담으며 반복	b06
`zip(A, B)` / `enumerate(A)`	두 목록을 짝지어 반복 / 0, 1, 2 … 번호를 붙여 반복	b23
`try: … except RuntimeError as e:`	오류가 나도 멈추지 않고 except로 넘어감. e에 오류 메시지가 담김	b12, b13, b16, b35
`def 이름() -> torch.device:`	함수 만들기. -> 뒤는 돌려주는 값의 자료형 안내 (없어도 동작)	b20, b33, b35
`lambda: 식`	이름 없는 한 줄 함수. 계산을 바로 하지 않고 '포장'해 두었다가 나중에 실행	b35
`with … as 폴더:`	블록이 끝나면 자동으로 정리 (임시 폴더 지우기 등)	b32, b33
`t.item()` / `t.tolist()`	텐서 → 파이썬 숫자 하나 / 파이썬 리스트	b08, b23, b35
`1_000_000` `1024 ** 2`	_ 는 자릿수 구분(값은 100만), ** 는 거듭제곱 (1024^2 = 1MB의 바이트 수)	b25, b32

1 텐서 만들기

b01 왜 텐서 · b02 b03 만들기 · b04 b24 난수 · b05 NumPy

b01 왜 텐서인가: 사진 한 장을 밝게

```
import time      # 시간 관련 도구 모음
import torch

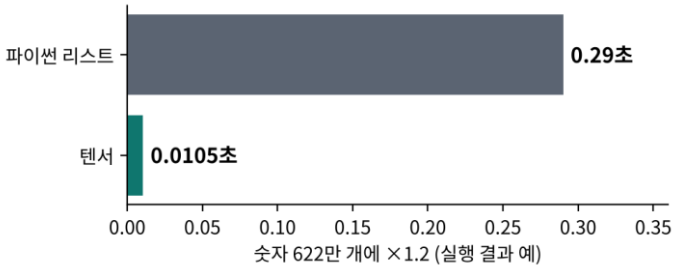
리스트_사진 = [0.5] * (1080 * 1920 * 3)    # 0.5를 622만 번 반복
텐서_사진 = torch.full((1080, 1920, 3), 0.5)  # 같은 크기 텐서

시작 = time.perf_counter()       # 스톱워치: 지금 시각(초)
밝게 = [값 * 1.2 for 값 in 리스트_사진]    # 하나씩 곱함
리스트_시간 = time.perf_counter() - 시작   # 지금 - 시작 = 걸린 초

시작 = time.perf_counter()
밝게_텐서 = 텐서_사진 * 1.2                # 한 번에 곱함
텐서_시간 = time.perf_counter() - 시작
print(f"리스트: {리스트_시간:.3f}초, 텐서: {텐서_시간:.4f}초")
```

```
숫자 개수: 6,220,800개
리스트: 0.290초, 텐서: 0.0105초    (실행 결과 예, 컴퓨터마다 다름)
텐서가 약 28배 빠름
```

처음 보는 문법	뜻
`time.perf_counter()`	스톱워치: 지금 시각(초). 끝 − 시작 = 걸린 시간
`[식 for 값 in 목록]`	하나씩 꺼내 계산한 새 리스트 (for문 한 줄 버전)
`f"{x:.3f}"` `f"{n:,}"`	{ } 안 값을 글자로. :.3f 소수 셋째, :, 쉼표



왜 빠를까

리스트: 숫자를 하나씩 꺼내 곱함
(622만 번 반복)
텐서: 숫자를 메모리에 빈틈없이 놓고
C로 짠 코드가 한꺼번에 계산

텐서 = 같은 자료형 숫자의 n차원 그릇

사진 한 장 = 1080 × 1920 × 색 3
→ 3차원 텐서
딥러닝의 입력·가중치·출력은 모두 텐서

b02·b03 텐서 만들기: 목록에서, 모양만으로

```
기온 = torch.tensor([12.5, 14.1, 13.0])   # 실수
인원 = torch.tensor([3, 5, 2])            # 정수
합격 = torch.tensor([True, False, True])  # 참/거짓
print(기온, 기온.dtype)
```

```
tensor([12.5000, 14.1000, 13.0000]) torch.float32
tensor([3, 5, 2]) torch.int64
tensor([ True, False,  True]) torch.bool
```

자료형은 넣은 값을 보고 정해짐

실수 → float32 (딥러닝 기본)
정수 → int64 (클래스 번호, 개수)
True/False → bool (마스크, b10)

```
torch.zeros(2, 3)          # 0으로 채운 2행 3열
torch.ones(2, 3)           # 1로 채우기
torch.full((2, 3), 7.0)    # 원하는 값으로
torch.arange(0, 10, 2)     # range처럼
torch.linspace(0, 1, 5)    # 같은 간격 5개
torch.eye(3)               # 단위 행렬
```

함수	결과
zeros(2, 3) / ones(2, 3)	0 또는 1로 채운 (2, 3)
full((2, 3), 7.0)	7로 채운 (2, 3), 모양은 괄호로
arange(0, 10, 2)	[0, 2, 4, 6, 8] 끝(10) 미포함
linspace(0, 1, 5)	[0, 0.25, 0.5, 0.75, 1] 끝 포함
eye(3)	대각선만 1인 (3, 3)

arange vs linspace

arange: 간격을 정함, 끝 미포함
linspace: 개수를 정함, 끝 포함

b04·b24 난수와 씨앗: 같은 결과를 다시 얻기

```
torch.manual_seed(42)         # 씨앗 심기
print(torch.rand(2, 3))       # 0 이상 1 미만
print(torch.randn(2, 3))      # 평균 0, 표준편차 1
print(torch.randint(1, 7, (5,)))  # 주사위 5번

torch.manual_seed(42)
print(torch.rand(2, 3))       # 처음과 똑같다
```

```
tensor([[0.8823, 0.9150, 0.3829],
        [0.9593, 0.3904, 0.6009]])
tensor([[ 1.1561,  0.3965, -2.4661],
        [ 0.3623,  0.3765, -0.1808]])
tensor([4, 5, 2, 3, 2])
tensor([[0.8823, 0.9150, 0.3829],     ← 같음
        [0.9593, 0.3904, 0.6009]])
```

함수	뽑는 범위
rand	0 이상 1 미만, 고르게
randn	정규분포 (평균 0, 표준편차 1)
randint(1, 7, (5,))	1~6 정수 5개 (7 미포함)

```
torch.manual_seed(42); 첫번째 = torch.rand(3)
torch.manual_seed(42)
torch.randn(5)           # 중간에 난수를 먼저 씀
두번째 = torch.rand(3)

생성기 = torch.Generator().manual_seed(7)
print(torch.rand(3, generator=생성기))
torch.randn(5)           # 전역 난수를 써도
생성기 = torch.Generator().manual_seed(7)
print(torch.rand(3, generator=생성기))
```

```
첫번째: [0.8823, 0.9150, 0.3829]
두번째: [0.8694, 0.5677, 0.7411]  ← 달라짐
생성기: [0.5349, 0.1988, 0.6592]  두 번 모두 같음
randperm(10, 생성기 씨앗 1): [5, 6, 1, 2, 0, 8, 9, 3, 7, 4]
```

씨앗은 '순서'를 고정함

씨앗이 같아도 중간에 난수를 한 번 더 쓰면 뒤 값이 바뀜
따로 만든 Generator는 전역 난수와 상관없이 자기 순서를 지킴

b05 NumPy와 오가기: 복사냐 공유냐

```
배열 = np.array([1.0, 2.0, 3.0])
복사본 = torch.tensor(배열)        # 새 그릇에 옮겨 담기
공유본 = torch.from_numpy(배열)    # 같은 그릇을 함께 쓰기

배열[0] = 100.0                    # NumPy 배열을 바꾸면?
print("NumPy  :", 배열)
print("복사본 :", 복사본)
print("공유본 :", 공유본)
print("다시 NumPy로:", 공유본.numpy())
```

```
NumPy  : [100.   2.   3.]
복사본 : tensor([1., 2., 3.], dtype=torch.float64)
공유본 : tensor([100.,   2.,   3.], dtype=torch.float64)
다시 NumPy로: [100.   2.   3.]
```

방법	메모리	원본을 바꾸면
torch.tensor(배열)	복사	그대로
torch.from_numpy(배열)	**공유**	같이 바뀜
텐서.numpy()	**공유**	같이 바뀜 (CPU 텐서만)

dtype이 float64인 이유

NumPy 기본 실수가 float64라 그대로 따라옴
모델(float32)에 넣기 전에 .float()로 바꿔 줌 (b35 자료형 오류)

공유는 빠르지만 조심

큰 배열을 복사 없이 바로 텐서로 쓸 수 있음
대신 한쪽을 고치면 다른 쪽도 바뀜 → 따로 쓰려면 torch.tensor() 또는 .clone()

확인 문제: 텐서 만들기

01 사진 두 장 (2, 1080, 1920, 3)이면 숫자는 몇 개?
흑백 한 장 (1080, 1920)은?

```
torch.full((2, 1080, 1920, 3), 0.5).numel()
```

02 인원을 [3, 5, 2.0]으로 바꾸면 dtype은?

```
인원 = torch.tensor([3, 5, 2.0])
```

03 arange(0, 1, 0.25)와 linspace(0, 1, 5)의 결과는?

```
torch.arange(0, 1, 0.25)
```

04 씨앗 42를 심고 rand(6)을 하면?
처음 rand(2, 3)과 비교

```
torch.manual_seed(42); torch.rand(6)
```

24 두 번째 블록에서 torch.randn(5) 줄을 지우면?

```
# torch.randn(5)
```

05 마지막 줄에 공유본[1] = 50 을 추가하면 배열은?

```
공유본[1] = 50
print(배열)
```

확인 문제 정답: 텐서 만들기

01
1080×1920×3 = 6,220,800 → 두 장 12,441,600개
흑백 한 장은 2,073,600개
계산 코드는 여전히 한 줄

02
torch.float32 → tensor([3., 5., 2.])
하나라도 실수면 전체가 실수

03
arange [0, 0.25, 0.5, 0.75] 4개 (1 미포함)
linspace [0, 0.25, 0.5, 0.75, 1.0] 5개 (끝 포함)

04
[0.8823, 0.9150, 0.3829, 0.9593, 0.3904, 0.6009] 같은 숫자가 한 줄로
모양과 상관없이 나오는 순서가 같음

24
두번째도 [0.8823, 0.9150, 0.3829] → 첫번째와 같아짐

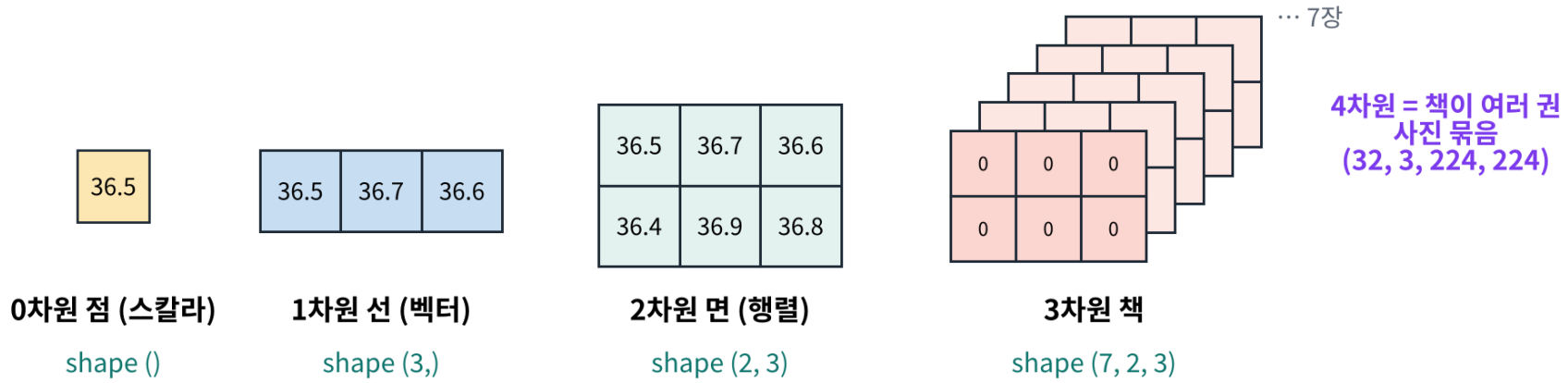
05
[100., 50., 3.] 공유는 양방향
텐서를 고쳐도 NumPy가 바뀜

2

모양과 자료형

b06 차원 · b07 사진 묶음 · b08 자료형 · b25 메모리

개념: 차원 = 숫자를 묶는 단계



속성	뜻	사진 묶음 (32, 3, 224, 224)의 값
t.ndim	차원 수	4
t.shape	차원마다 크기, 바깥 → 안쪽	torch.Size([32, 3, 224, 224])
t.numel()	전체 숫자 개수 (shape를 다 곱함)	4,816,896
t[0]	맨 앞 차원에서 하나 꺼냄	(3, 224, 224) 사진 한 장

b06·b07 ndim, shape, numel 읽기

```
점 = torch.tensor(36.5)                 # 체온 하나
선 = torch.tensor([36.5, 36.7, 36.6])   # 하루 세 번
면 = torch.tensor([[36.5, 36.7, 36.6],
                   [36.4, 36.9, 36.8]])  # 이틀치
책 = torch.zeros(7, 2, 3)               # 일주일치 표 7장
# (이름, 텐서) 짝을 하나씩 꺼내 이름, t에 나눠 담음
for 이름, t in [("점", 점), ("선", 선), ("면", 면), ("책", 책)]:
    print(f"{이름}: ndim={t.ndim}, shape={tuple(t.shape)}, 숫자 {t.numel()}개")
```

```
점: ndim=0, shape=(), 숫자 1개
선: ndim=1, shape=(3,), 숫자 3개
면: ndim=2, shape=(2, 3), 숫자 6개
책: ndim=3, shape=(7, 2, 3), 숫자 42개
```

shape 읽는 법

(7, 2, 3) = 숫자 3개짜리 줄이 2개 모인 표가 7장
바깥(큰 묶음)부터 안쪽 순서
숫자 개수 = 7 × 2 × 3

```
사진_묶음 = torch.zeros(32, 3, 224, 224)
print(사진_묶음.shape)
print(f"숫자 {사진_묶음.numel():,}개, 차원 {사진_묶음.ndim}개")
print("첫 사진의 모양:", 사진_묶음[0].shape)
```

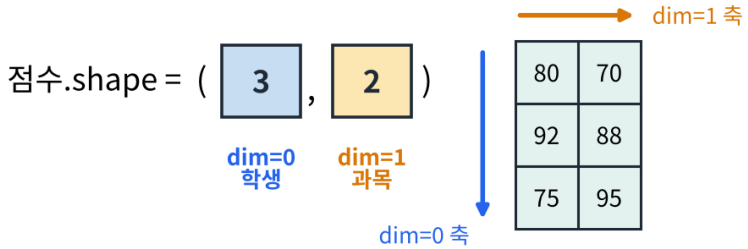
```
torch.Size([32, 3, 224, 224])
숫자 4,816,896개, 차원 4개
첫 사진의 모양: torch.Size([3, 224, 224])
```

자리	뜻	크기
N	묶음 (사진 장수)	32
C	채널 (빨강·초록·파랑)	3
H	세로 픽셀	224
W	가로 픽셀	224

PyTorch 이미지는 (N, C, H, W): 채널이 앞 (b13 permute)

dim은 무엇 기준? shape의 자리 번호

dim = shape에서 몇 번째 자리인지 (왼쪽부터 0, 1, 2, …)

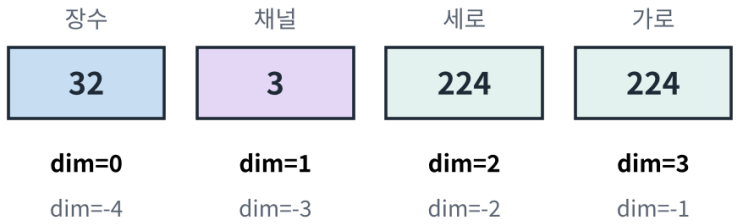


sum(dim=0) → 3이 사라짐 → (2,) 과목별 합 [247, 253]

sum(dim=1) → 2가 사라짐 → (3,) 학생별 합 [150, 180, 170]

외우는 법: dim=k → shape의 k번째 숫자가 사라짐

사진 묶음.shape = (32, 3, 224, 224)



음수는 뒤에서부터: dim=-1 = 맨 뒤(가로)

mean(dim=1) → 채널이 사라짐 → (32, 224, 224) 흑백처럼

dim=0이 항상 '세로'는 아님

2차원 표에서는 dim=0이 학생(세로) 쪽이지만, 사진 묶음에서는 dim=0이 장수
헷갈리면 print(t.shape)로 모양을 보고, 사라질 숫자의 자리 번호를 dim에 넣기

b08 자료형(dtype): 섞이면 넓은 쪽으로

```
a = torch.tensor([1, 2, 3])
b = torch.tensor([0.5, 0.5, 0.5])
print((a + b).dtype)               # 정수 + 실수 → 실수
print(torch.tensor([7, 8]) / 2)    # / 는 항상 실수
print(torch.tensor([7, 8]) // 2)   # // 는 몫, 정수

x = torch.tensor([1.5, 2.7])
print(x.int(), x.to(torch.int64).dtype)  # 소수점 버림
print(x.half().dtype, x.double().dtype)

print(torch.tensor(0.1) + torch.tensor(0.2))
print((torch.tensor(0.1) + torch.tensor(0.2)).item())
print((torch.tensor(0.1, dtype=torch.float64)
       + torch.tensor(0.2, dtype=torch.float64)).item())
```

자료형	칸당	주로 쓰는 곳
float32	4B	기본 실수: 데이터, 가중치
float64	8B	NumPy 기본 (b05)
float16 / bfloat16	2B	GPU 메모리 절약 (b25)
int64	8B	클래스 번호, 인덱스
bool	1B	마스크 (b10)

```
torch.float32
tensor([3.5000, 4.0000])
tensor([3, 4])
tensor([1, 2], dtype=torch.int32) torch.int64
torch.float16 torch.float64
tensor(0.3000)
0.30000001192092896
0.30000000000000004
```

실수는 근삿값

0.1, 0.2는 2진수로 딱 떨어지지 않음
float32는 유효숫자 약 7자리 → 화면엔 0.3000,
실제는 0.30000001
비교는 == 대신 torch.isclose

x.int(): 반올림이 아니라 버림 (1.5 → 1, 2.7 → 2)

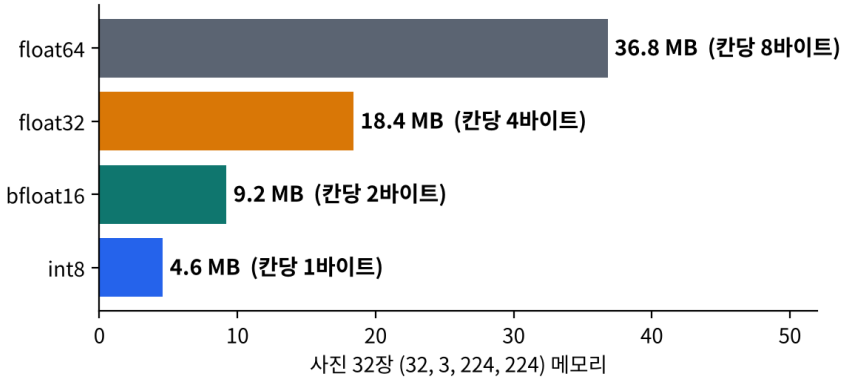
b25 메모리 크기 = 칸당 바이트 × 숫자 개수

```
MB = 1024 ** 2            # 1024² 바이트 = 1MB
사진묶음 = torch.zeros(32, 3, 224, 224)
print(사진묶음.element_size(), "바이트 ×",
      f"{사진묶음.numel():,}개")
print(f"{사진묶음.element_size() * 사진묶음.numel() / MB:.1f} MB")

for 자료형 in [torch.float64, torch.float32,
               torch.bfloat16, torch.int8]:
    t = torch.zeros(32, 3, 224, 224, dtype=자료형)
    크기 = t.element_size() * t.numel() / MB

GB = 1024 ** 3            # 1024³ 바이트 = 1GB
# 7_000_000_000: _ 는 자릿수 구분 (= 70억)
print(f"70억 × 4바이트 = {7_000_000_000 * 4 / GB:.1f} GB")
```

```
4 바이트 × 4,816,896개
18.4 MB
torch.float64   칸당 8바이트 →   36.8 MB
torch.float32   칸당 4바이트 →   18.4 MB
torch.bfloat16  칸당 2바이트 →    9.2 MB
torch.int8      칸당 1바이트 →    4.6 MB
70억 × 4바이트 = 26.1 GB
```



계산해 보기

사진 32장: 4,816,896 × 4바이트 ÷ 1024² = 18.4 MB
70억 파라미터 모델: float32면 26.1 GB, bfloat16이면 13.0 GB

확인 문제: 모양과 자료형

06 torch.zeros(4, 7, 2, 3)의 ndim과 숫자 개수는?

```
t = torch.zeros(4, 7, 2, 3)
```

07 흑백 28×28 사진 64장 묶음의 shape와 숫자 개수는?

```
torch.zeros(64, ?, 28, 28)
```

08 torch.tensor([7, 8]) / 2 와 // 2의 dtype은?

```
(torch.tensor([7, 8]) / 2).dtype
```

25 사진 묶음을 float16으로 바꾸면 몇 MB?
70억 모델을 bfloat16으로 담으면?

```
dtype=torch.float16
```

확인 문제 정답: 모양과 자료형

06 ndim 4, numel 168 (4 × 7 × 2 × 3)
일주일치 표 7장이 4주치

07 (64, 1, 28, 28), 50,176개
흑백이어도 채널 차원 1을 남겨 둠

08 / → float32 [3.5, 4.0]
// → int64 [3, 4] (몫)

25 9.2 MB (칸당 2바이트)
70억 × 2 ÷ 1024^3 ≈ 13.0 GB (float32의 절반)

3 인덱싱과 슬라이싱

b09 인덱싱 · b10 마스크 · b11 공유 · b26 where · b27 gather · b28 scatter

b09 인덱싱: [행, 열]로 꺼내기

기온[0]

2	13.5	25.7	15
4.5	15	25	17.5
1	14	26	15.5

기온[0, 2]

2	13.5	25.7	15
4.5	15	25	17.5
1	14	26	15.5

기온[:, 2]

2	13.5	25.7	15
4.5	15	25	17.5
1	14	26	15.5

기온[1:, :2]

2	13.5	25.7	15
4.5	15	25	17.5
1	14	26	15.5

기온[-1]

2	13.5	25.7	15
4.5	15	25	17.5
1	14	26	15.5

행 = 도시 3곳, 열 = 분기 4개

```
기온 = torch.tensor([[ 2.0, 13.5, 25.7, 15.0],   # 도시 3곳
                     [ 4.5, 15.0, 25.0, 17.5],   # × 분기 4개
                     [ 1.0, 14.0, 26.0, 15.5]])
print(기온[0])        # 첫 도시의 네 분기
print(기온[0, 2])     # 첫 도시의 3분기
print(기온[:, 2])     # 모든 도시의 3분기
print(기온[1:, :2])   # 둘째 도시부터, 1~2분기
print(기온[-1])       # 마지막 도시
```

```
tensor([ 2.0000, 13.5000, 25.7000, 15.0000])
tensor(25.7000)
tensor([25.7000, 25.0000, 26.0000])
tensor([[ 4.5000, 15.0000],
        [ 1.0000, 14.0000]])
tensor([ 1.0000, 14.0000, 26.0000, 15.5000])
```

b10 마스크와 목록으로 고르기

```
더운가 = 기온 > 20          # 칸마다 True/False
print(더운가)
print(기온[더운가])          # 참인 칸만, 1차원으로
print(기온[[0, 2]])          # 0번, 2번 도시만

얼음 = 기온.clone()
얼음[얼음 < 5] = 0.0         # 5도 미만은 0으로
print(얼음)
```

```
tensor([[False, False,  True, False],
        [False, False,  True, False],
        [False, False,  True, False]])
tensor([25.7000, 25.0000, 26.0000])
tensor([[ 2.0000, 13.5000, 25.7000, 15.0000],
        [ 1.0000, 14.0000, 26.0000, 15.5000]])
tensor([[ 0.0000, 13.5000, 25.7000, 15.0000],
        [ 0.0000, 15.0000, 25.0000, 17.5000],
        [ 0.0000, 14.0000, 26.0000, 15.5000]])
```

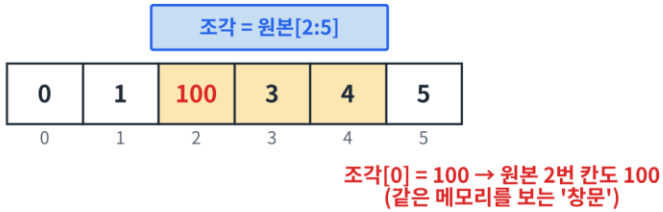
고르는 방법	예	결과	원본과
숫자	기온[0]	차원 하나 줄어듦	공유
슬라이스	기온[1:, :2]	차원 유지	공유 (b11)
목록	기온[[0, 2]]	고른 줄만	복사
마스크	기온[기온 > 20]	1차원으로 펴짐	복사

마스크에 대입

얼음[얼음 < 5] = 0 은 조건에 맞는 칸만 바꿈
원본을 지키려고 먼저 clone()

b11 슬라이스는 원본을 공유함

원본 = torch.arange(6)



따로 쓰려면
원본[2:5].clone()

연산	원본과 메모리 공유?
슬라이스 x[2:5], x[0]	**공유**
view, .T, permute, unsqueeze	**공유**
reshape	되도록 공유, 안 되면 복사
목록·마스크 인덱싱, clone()	복사

```
원본 = torch.arange(6)
조각 = 원본[2:5]       # 복사하지 않음
조각[0] = 100
print("조각:", 조각)
print("원본:", 원본)    # 원본도 바뀜!

안전 = 원본[2:5].clone()
안전[0] = -1
print("원본:", 원본)    # 이번엔 그대로
```

```
조각: tensor([100,   3,   4])
원본: tensor([  0,   1, 100,   3,   4,   5])
원본: tensor([  0,   1, 100,   3,   4,   5])
```

왜 공유할까

큰 텐서를 자를 때마다 복사하면 느리고 메모리 낭비
대신 고치면 원본도 바뀜 → 따로 쓸 땐 clone()

b26 where와 clamp: 조건으로 값 고르기

```
print(torch.where(기온 > 20, 1, 0))
print(torch.where(기온 < 5, 0.0, 기온))  # 5도 미만만 0
print(기온.clamp(min=5.0, max=20.0))
print(torch.tensor([-0.3, 0.5, 1.7]).clamp(0.0, 1.0))
print(torch.tensor([-2.0, 0.0, 3.0]).clamp(min=0.0))
```

```
tensor([[0, 0, 1, 0],
        [0, 0, 1, 0],
        [0, 0, 1, 0]])
tensor([[ 0.0000, 13.5000, 25.7000, 15.0000],
        [ 0.0000, 15.0000, 25.0000, 17.5000],
        [ 0.0000, 14.0000, 26.0000, 15.5000]])
tensor([[ 5.0000, 13.5000, 20.0000, 15.0000],
        [ 5.0000, 15.0000, 20.0000, 17.5000],
        [ 5.0000, 14.0000, 20.0000, 15.5000]])
tensor([0.0000, 0.5000, 1.0000])
tensor([0., 0., 3.])
```

코드	하는 일
where(기온 > 20, 1, 0)	더운 칸 표시 (0/1)
where(기온 < 5, 0.0, 기온)	b10 얼음과 같은 결과, 원본은 그대로
clamp(5, 20)	5보다 작으면 5, 20보다 크면 20
clamp(0, 1)	확률·픽셀 값을 0~1로 묶기
clamp(min=0)	음수만 0 = ReLU와 같은 계산

마스크 대입 vs where

b10 얼음[얼음 < 5] = 0 → 그 텐서를 직접 고침
where → 새 텐서를 만들어 돌려줌

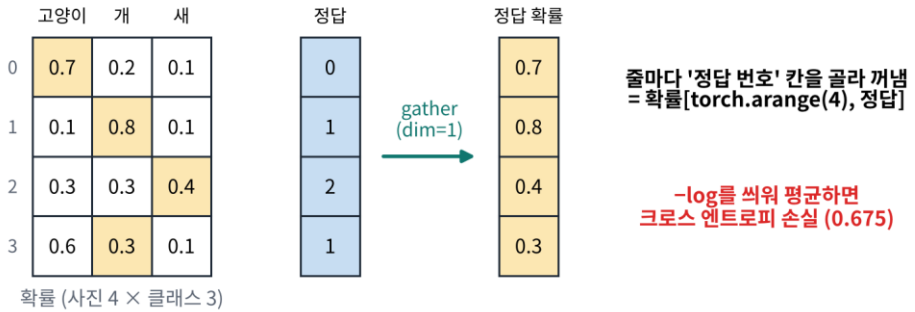
읽는 법: torch.where(조건, A, B)

칸마다 '조건이 참이면 A, 아니면 B'
파이썬 A if 조건 else B를 모든 칸에 한 번에
예) where(기온 > 20, 1, 0): 25.7 → 1, 13.5 → 0

읽는 법: t.clamp(min, max)

바닥(min)과 천장(max) 사이로 밀어 넣기
min보다 작으면 min, max보다 크면 max
예) clamp(5, 20): 2.0 → 5, 13.5 → 13.5, 25.7 → 20

b27 gather: 줄마다 정답 칸의 확률 꺼내기



줄마다 '정답 번호' 칸을 골라 꺼냄
= 확률[torch.arange(4), 정답]

−log를 씌워 평균하면
크로스 엔트로피 손실 (0.675)

```
확률 = torch.tensor([[0.7, 0.2, 0.1],
                     [0.1, 0.8, 0.1],
                     [0.3, 0.3, 0.4],
                     [0.6, 0.3, 0.1]])
정답 = torch.tensor([0, 1, 2, 1])
예측 = 확률.argmax(dim=1)
print(확률[torch.arange(4), 정답])

번호표 = 정답.unsqueeze(1)       # (4,) → (4, 1)
정답_확률 = 확률.gather(dim=1, index=번호표)
print(정답_확률.squeeze(1))
print(정답_확률.log().neg().mean())   # -log 평균
```

```
예측: tensor([0, 1, 2, 0]) 정확도: 0.75
tensor([0.7000, 0.8000, 0.4000, 0.3000])
torch.Size([4, 1])
tensor([0.7000, 0.8000, 0.4000, 0.3000])
tensor(0.6750)
```

gather(dim=1, index)

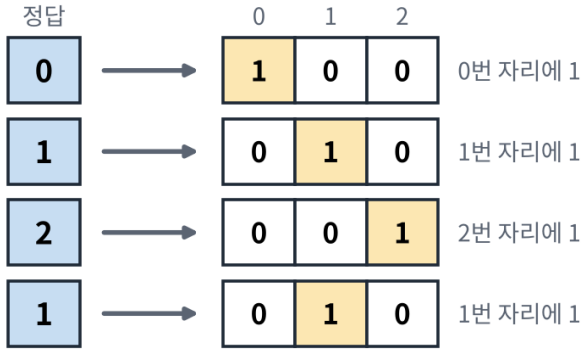
줄마다 index가 가리키는 칸을 꺼냄
index는 확률과 차원 수가 같아야 해서 (4,) → unsqueeze(1) → (4, 1)
결과도 (4, 1)이라 squeeze(1)로 폄

크로스 엔트로피와 연결

−log(정답 확률)의 평균 = 크로스 엔트로피 손실 0.675
정답 확률이 낮은 4번째(0.3)가 손실을 키움

그림으로 보기: one_hot과 bincount

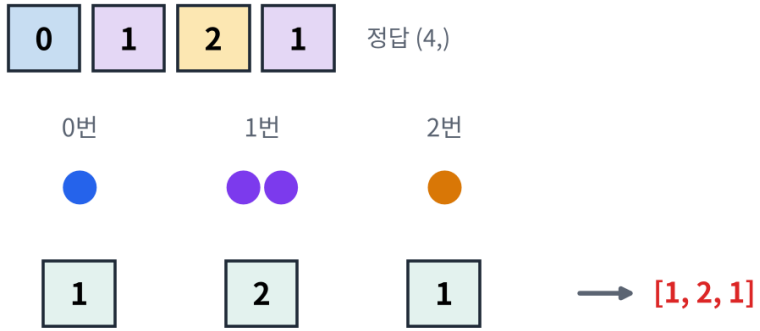
① one_hot(정답, 3): 번호 → 그 자리만 1인 줄



(4,) → (4, 3) 칸 수 = num_classes

결과는 정수(int64) → 계산에 쓰려면 .float()

② bincount(정답): 번호마다 개수 세기



결과 길이 = 가장 큰 번호 + 1 (0번부터 칸이 생김)

예) bincount([2, 2, 0]) → [1, 0, 2] 한 번도 없는 1번도 0으로 칸이 있음

```
정답 = torch.tensor([0, 1, 2, 1])
torch.nn.functional.one_hot(정답, num_classes=3)  # (4,)→(4, 3)
torch.bincount(정답)                 # 번호별 개수
```

```
tensor([[1, 0, 0], [0, 1, 0], [0, 0, 1], [0, 1, 0]])
tensor([1, 2, 1])
```

b28 scatter: 번호 자리에 값 쓰기

① scatter_: 번호가 가리키는 칸에 값 쓰기 → 원핫

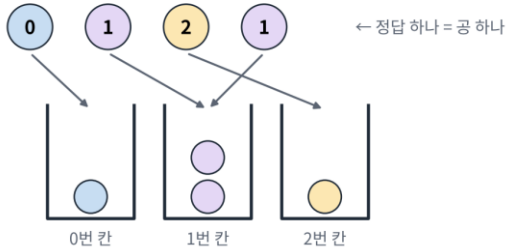


i번 줄의
정답[i]번 칸에 1
나머지는 0 그대로

= one_hot(정답, 3)

gather는 번호 칸에서 '꺼내기', scatter는 번호 칸에 '넣기'

② scatter_add_: 같은 칸이면 더하기 → 개수 세기



개수 [1, 2, 1] = bincount(정답)

함수	하는 일
gather	번호 칸에서 꺼냄 (읽기)
scatter_	번호 칸에 씀 (쓰기)
scatter_add_	같은 칸이면 더함 → 개수 세기
one_hot	원핫 지름길 (결과는 정수)
bincount	번호별 개수 지름길

```
정답 = torch.tensor([0, 1, 2, 1])
원핫 = torch.zeros(4, 3)                    # 0으로 채운 빈 표
원핫.scatter_(dim=1, index=정답.unsqueeze(1), value=1.0)
#  줄마다 정답 번호 칸(dim=1 방향)에 1을 씀
print(원핫)
print(torch.nn.functional.one_hot(정답, num_classes=3))  # 지름길

개수 = torch.zeros(3).scatter_add_(0, 정답, torch.ones(4))
#  공(1) 4개를 정답 번호 칸에 차례로 더함
print(개수)
print(torch.bincount(정답))                 # 지름길
```

```
tensor([[1., 0., 0.],
        [0., 1., 0.],
        [0., 0., 1.],
        [0., 1., 0.]])
tensor([[1, 0, 0], [0, 1, 0],
        [0, 0, 1], [0, 1, 0]])
tensor([1., 2., 1.])
tensor([1, 2, 1])
```

one_hot 결과는 정수 → 계산에 쓰려면 .float()

확인 문제: 인덱싱과 슬라이싱

09 기온[1:, 1:3]의 결과는?

```
print(기온[1:, 1:3])
```

10 15도 이상인 칸만 고르면?

```
기온[기온 >= 15]
```

11 조각[0] = 100 대신 원본[3] = 50 을 하면 조각은?

```
조각 = 원본[2:5]
원본[3] = 50
```

26 where(기온 > 20, 기온, 20.0)과 기온.clamp(min=20)은 같을까?

```
torch.equal(A, B)
```

27 정답을 [0, 1, 2, 2]로 바꾸면 정답 확률과 손실은?

```
정답 = torch.tensor([0, 1, 2, 2])
```

28 정답이 [2, 2, 0]이면 one_hot과 bincount는?

```
정답 = torch.tensor([2, 2, 0])
```

확인 문제 정답: 인덱싱과 슬라이싱

09
[[15.0, 25.0], [14.0, 26.0]]
둘째 도시부터
2~3분기 (3 미포함)

10
[25.7, 15.0, 15.0, 25.0, 17.5, 26.0, 15.5]
7개
행 순서대로 1차원

11
[2, 50, 4]
공유는 양방향: 원본을 고쳐도 조각이 바뀜

26
같음 (True)
20보다 작은 칸을 모두 20으로 올리는 같은 계산

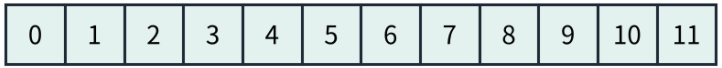
27
[0.7, 0.8, 0.4, 0.1], 손실 0.9497
마지막 정답 확률이 0.1이라 손실이 커짐

28
one_hot [[0,0,1], [0,0,1], [1,0,0]]
bincount [1, 0, 2]

4 모양 바꾸기

b12 reshape · b13 permute · b14 squeeze·cat·stack · b29 stride

b12 reshape: 줄 바꾸는 위치만 바꾸기



arange(12): 메모리에 한 줄로



reshape(3, 4)



reshape(2, -1) → (2, 6)

읽는 순서(0, 1, 2, …)는 그대로,
줄 바꾸는 위치만 달라짐

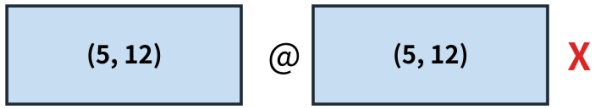
개수가 맞아야 함: 12 = 3×4 = 2×6
reshape(5, -1) → 오류 (12÷5 안 나눠짐)

```
x = torch.arange(12)
print(x.reshape(3, 4))
print(x.reshape(2, -1))       # -1: 알아서 6
print(x.reshape(2, 2, 3).shape)
try:                          # 오류가 나도 멈추지 않게
    x.reshape(5, -1)
except RuntimeError as e:  # 오류면 여기로, e = 메시지
    print("오류:", e)
```

```
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
tensor([[[ 0,  1,  2],
         [ 3,  4,  5]],
        [[ 6,  7,  8],
         [ 9, 10, 11]]])
torch.Size([2, 2, 3])
오류: shape '[5, -1]' is invalid for input of size 12
```

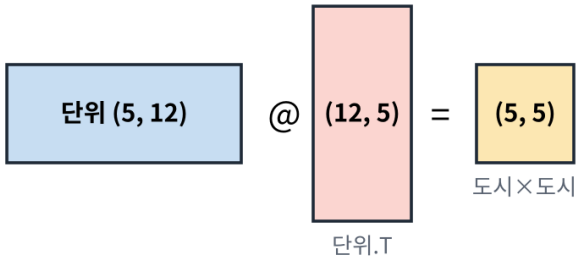
전치(.T)는 왜 하나

① 행렬 곱 모양 맞추기 (b23)

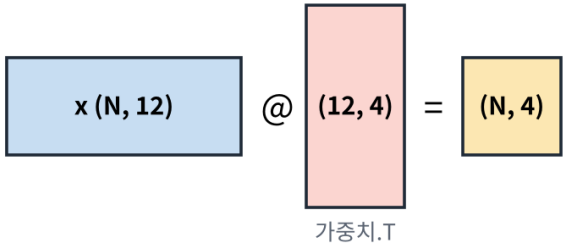


안쪽 12 vs 5 → 오류

한쪽을 세우면 안쪽 12 = 12 → OK



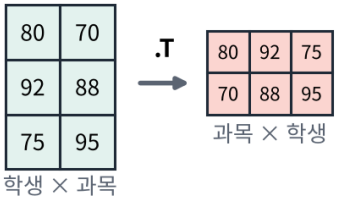
② nn.Linear(12, 4) 안의 계산



가중치는 (출력 4, 입력 12)로 저장됨

→ 입력 @ 가중치.T + 편향

③ 보는 기준 바꾸기



한 줄 = 한 과목

과목 단위로 다루기 편함

전치는 복사 없음 (b29)

공통점: 축 순서를 바꿔서 맞추기

행렬 곱의 안쪽 크기를 맞추거나, '한 줄'이 뜻하는 것을 바꿈
2차원은 .T, 3차원 이상은 permute (b13)

b13 permute와 전치: 차원 순서 바꾸기

```
사진 = torch.zeros(224, 224, 3)          # (세로, 가로, 색)
파이토치용 = 사진.permute(2, 0, 1)      # (색, 세로, 가로)
print(사진.shape, "→", 파이토치용.shape)

행렬 = torch.arange(6).reshape(2, 3)
print(행렬.T)                            # 2차원 전치
print(행렬.T.is_contiguous())
try:                          # 오류가 나도 계속 진행
    행렬.T.view(6)
except RuntimeError as e:
    print("view 오류:", str(e).split(".")[0])  # 첫 문장만
print(행렬.T.reshape(6))
```

```
torch.Size([224, 224, 3]) → torch.Size([3, 224, 224])
tensor([[0, 3],
        [1, 4],
        [2, 5]])
False
view 오류: view size is not compatible with input
  tensor's size and stride (...)
tensor([0, 3, 1, 4, 2, 5])
```

함수	하는 일
permute(2, 0, 1)	차원 순서를 원하는 대로 (복사 없음)
.T	2차원 전치 = transpose(0, 1)
view	메모리가 이어져 있을 때만, 항상 공유
reshape	되면 view, 안 되면 복사해서라도

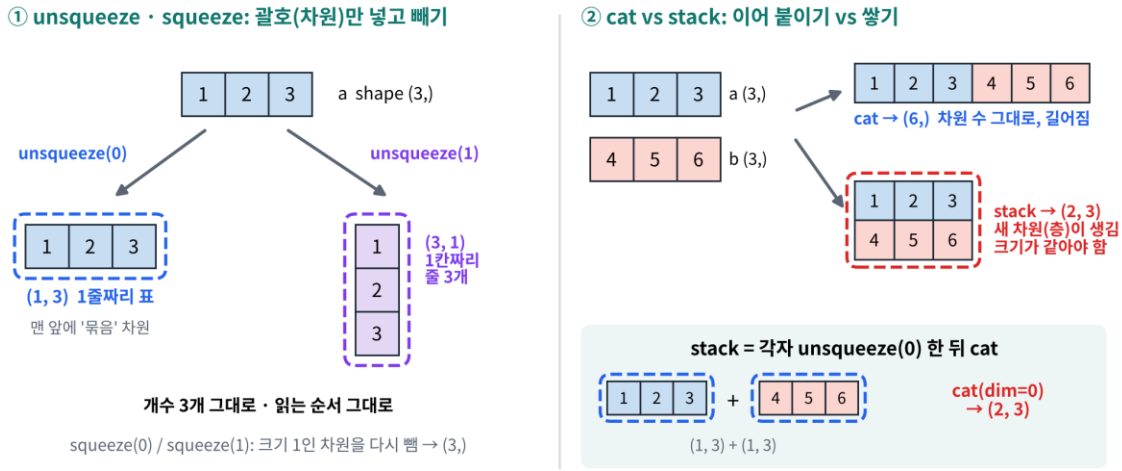
왜 view가 실패할까

전치는 메모리를 옮기지 않고 읽는 순서만 바꿈
0, 3, 1, 4, 2, 5는 메모리에서 띄엄띄엄 → 한 줄로 이어 볼 수 없음 (b29)

사진 순서

PIL·OpenCV·NumPy 사진은 (H, W, C), PyTorch 모델은 (C, H, W)

그림으로 보기: unsqueeze와 cat·stack



① 왜? 묶음 차원

모델은 맨 앞 차원을
'묶음(장수)'으로 보고 계산
사진 한 장 (3, 32, 32)도
(1, 3, 32, 32)로 넣어야 함

② 왜? 방향 맞추기

(3,) → (3, 1)로 세우면
학생별 브로드캐스팅 (b16)
gather 번호표 (b27)

```
모델 = nn.Sequential(nn.Flatten(), nn.Linear(3 * 32 * 32, 10))
사진 = torch.zeros(3, 32, 32)          # 사진 한 장

모델(사진)                          # Flatten이 3을 '3장'으로 착각 → 오류
출력 = 모델(사진.unsqueeze(0))  # (1, 3, 32, 32): 1장짜리 묶음
print(출력.shape, 출력.squeeze(0).shape)   # 결과에서 묶음 빼기
```

```
오류: mat1 and mat2 shapes cannot be
  multiplied (3x1024 and 3072x10)
torch.Size([1, 10]) torch.Size([10])
```

b14 squeeze·unsqueeze, cat·stack, flatten

```
사진 = torch.zeros(3, 224, 224)
묶음 = 사진.unsqueeze(0)      # 맨 앞에 크기 1 추가
print(묶음.shape)
print(묶음.squeeze(0).shape)  # 크기 1 차원 빼기

a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])
print(torch.cat([a, b]))      # 이어 붙이기
print(torch.stack([a, b]))    # 쌓기: 새 차원
print(torch.stack([a, b]).shape,
      torch.stack([a, b], dim=1).shape)
print(torch.zeros(32, 3, 4, 4).flatten(start_dim=1).shape)
```

```
torch.Size([1, 3, 224, 224])
torch.Size([3, 224, 224])
tensor([1, 2, 3, 4, 5, 6])
tensor([[1, 2, 3],
        [4, 5, 6]])
torch.Size([2, 3]) torch.Size([3, 2])
torch.Size([32, 48])
```

함수	모양 변화	언제
unsqueeze(0)	(3,224,224) → (1,3,224,224)	사진 한 장을 모델에
squeeze(0)	크기 1 차원 제거	결과에서 묶음 빼기
cat	차원 수 그대로 길어짐	데이터 이어 붙이기
stack	새 차원 추가	묶음 만들기
flatten(1)	(32,3,4,4) → (32,48)	CNN → Linear

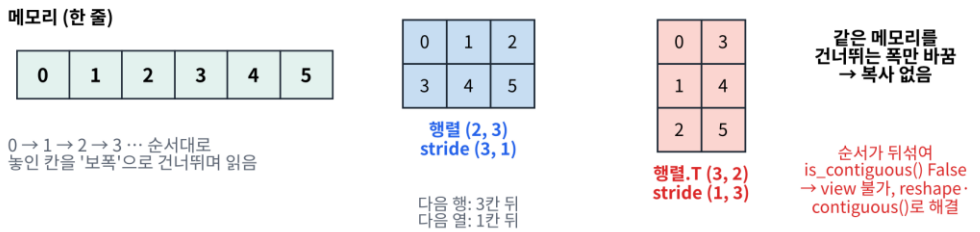
flatten(start_dim=1)

0번(묶음)은 두고 나머지만 한 줄로
3 × 4 × 4 = 48 → (32, 48)

stack은 크기가 같아야 함

[3]과 [4]를 쌓으면 오류 (b35)
크기가 다르면 cat으로 이어 붙이기

b29 stride: 같은 메모리를 다른 보폭으로 읽기




```
행렬 = torch.arange(6).reshape(2, 3)
print("보폭:", 행렬.stride())
전치 = 행렬.T
print("보폭:", 전치.stride())
print("같은 메모리?",
      전치.data_ptr() == 행렬.data_ptr())
print("이어져 있나?", 전치.is_contiguous())
정리본 = 전치.contiguous()    # 새로 늘어놓은 복사본
print(정리본.stride(), 정리본.is_contiguous())
print(정리본.data_ptr() == 행렬.data_ptr())
print(정리본.view(6))         # 이제 된다
```

```
보폭: (3, 1)
보폭: (1, 3)
같은 메모리? True
이어져 있나? False
(2, 1) True
False
tensor([0, 3, 1, 4, 2, 5])
```

	모양	stride	메모리	view(6)
행렬	(2, 3)	(3, 1)	원본	O
행렬.T	(3, 2)	(1, 3)	같은 메모리	X
.T.contiguous()	(3, 2)	(2, 1)	새로 복사	O

view란?

복사 없이 같은 메모리를 다른 모양으로 보는 것 (reshape의 복사 안 하는 버전)
메모리 순서대로 읽을 수 있을 때만 가능 → .T는 X
contiguous()로 다시 늘어놓거나 reshape 사용

확인 문제: 모양 바꾸기

12 x.reshape(-1, 3), reshape(3, -1, 2), reshape(4, -1, 2)의 결과는?

```
x = torch.arange(12)
```

13 사진 32장 (32, 224, 224, 3)을 (32, 3, 224, 224)로 바꾸려면?

```
사진들.permute(?, ?, ?, ?)
```

14 torch.zeros(32, 3, 4, 4).flatten()의 모양은?

```
torch.zeros(32, 3, 4, 4).flatten().shape
```

29 torch.zeros(4, 5)의 stride와 .T의 stride는?

```
torch.zeros(4, 5).stride()
```

확인 문제 정답: 모양 바꾸기

12

(4, 3) / (3, 2, 2) / 오류
4 × 2 = 8로 12가 나눠지지 않음

13

permute(0, 3, 1, 2)
묶음 0번은 그대로
채널(3번)을 둘째 자리로

14

(1536,) 묶음 차원까지 모두 펴짐
start_dim=1이면 (32, 48)로 사진마다 한 줄

29

(5, 1)과 (1, 5)
다음 행은 5칸 뒤
전치하면 숫자만 맞바뀜

5 한꺼번에 계산하기

b15 원소별 · b16 브로드캐스팅 · b17 행렬 곱 · b18 축 접기 · b19 제자리 · b30 einsum · b31 정렬

b15·b19 원소별 연산과 제자리 연산

```
국어 = torch.tensor([80.0, 92.0, 75.0])
수학 = torch.tensor([70.0, 88.0, 95.0])
print(국어 + 수학)          # 같은 자리끼리
print((국어 + 수학) / 2)    # 평균
print(국어 > 수학)          # 비교도 칸마다
print(torch.sqrt(국어))     # 함수도 칸마다
```

```
tensor([150., 180., 170.])
tensor([75., 90., 85.])
tensor([ True,  True, False])
tensor([8.9443, 9.5917, 8.6603])
```

원소별 연산

모양이 같은 두 텐서는 같은 자리끼리 계산
+, −, *, /, >, sqrt, exp, log 모두 칸마다 → 반복문 없이 한 줄 (b01이 빠른 이유)

```
a = torch.tensor([1.0, 2.0, 3.0])
b = a.add(10)      # 새 텐서를 돌려줌
print(a, b)
a.add_(10)         # 밑줄: a 자신을 바꿈
print(a)
```

```
tensor([1., 2., 3.]) tensor([11., 12., 13.])
tensor([11., 12., 13.])
```

쓰는 법	결과
a.add(10), a + 10	새 텐서, a는 그대로
a.add_(10), a += 10	a 자신이 바뀜 (제자리)

밑줄 _ = 제자리 연산

메모리를 아끼지만
학습 중 기울기 계산에 필요한 값을 덮어쓰면 오류
처음엔 안 쓰는 게 안전 (zero_grad, nn.init.zeros_ 같은 곳에서 봄)

브로드캐스팅 규칙: 왜 뒤 차원부터 맞추나



(2,)는 무엇?

숫자 2개짜리 줄 하나 = [3, 10]
행·열 구분이 없는 1차원
모자란 차원은 항상 '앞'에 1로 붙임
→ (1, 2): 1줄 2칸

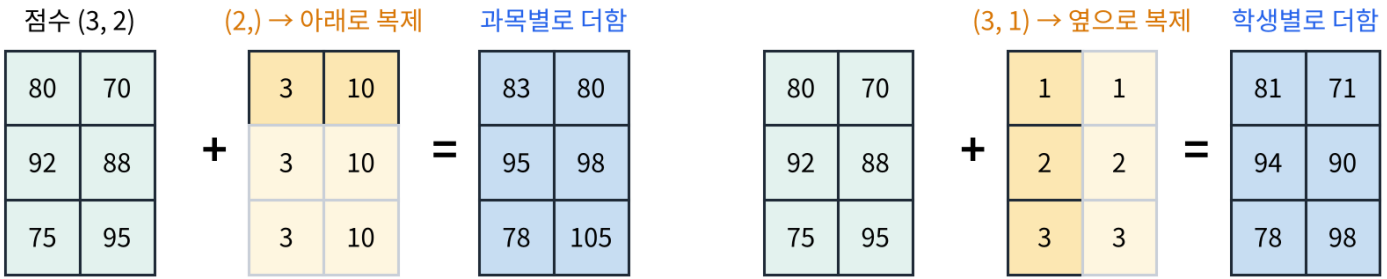
그래서 2는 '열'과 비교

(2,)의 2는 늘 맨 뒤(열) 자리
점수의 열도 2 → 맞음
(3,)이면 열 2 vs 3 → 오류

행마다 다른 값을 더하려면

(3,)을 (3, 1)로 세움
보너스.unsqueeze(1)

b16 브로드캐스팅: 모양이 달라도 늘려서 계산



규칙: 뒤 차원부터 맞춰 보고, 크기가 같거나 1이면 OK. (3, 2) + (3,)은 2 ≠ 3이라 오류

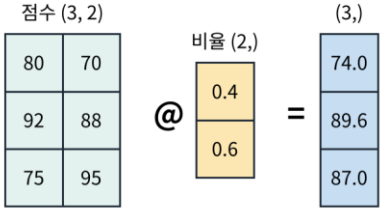
```
점수 = torch.tensor([[80.0, 70.0], [92.0, 88.0], [75.0, 95.0]])
print(점수 + 5)                              # 모두 5점
print(점수 + torch.tensor([3.0, 10.0]))      # (3,2)+(2,)
print(점수 + torch.tensor([[1.0], [2.0], [3.0]]))   # (3,2)+(3,1)
점수 + torch.tensor([1.0, 2.0, 3.0])        # (3,2)+(3,) → 오류
```

```
+5       → [[85, 75], [97, 93], [80, 100]]
+(2,)    → [[83, 80], [95, 98], [78, 105]]
+(3, 1) → [[81, 71], [94, 90], [78, 98]]
오류: The size of tensor a (2) must match the size
  of tensor b (3) at non-singleton dimension 1
```

(3, 2) + (3,)이 안 되는 이유

뒤에서 맞추면 2 vs 3 (둘 다 1이 아님) → 학생별로 더하려면 (3, 1)로 세움: unsqueeze(1)

b17 행렬 곱 @: 곱해서 더하기



첫 학생: 80×0.4 + 70×0.6 = 74.0

행 하나 × 비율 = 곱해서 더하기 (가중합)

모양 규칙: (3, 2) @ (2, ?) → 안쪽 2끼리 맞아야 함
= nn.Linear가 하는 계산 (6장)

```
비율 = torch.tensor([0.4, 0.6])
print(점수 @ 비율)                # (3,2)@(2,)
print((점수 * 비율).sum(dim=1))   # 풀어 쓰면

두_기준 = torch.tensor([[0.4, 0.5],
                        [0.6, 0.5]])  # 기준 A, B
print(점수 @ 두_기준)             # (3,2)@(2,2)
```

```
tensor([74.0000, 89.6000, 87.0000])
tensor([74.0000, 89.6000, 87.0000])
tensor([[74.0000, 75.0000],
        [89.6000, 90.0000],
        [87.0000, 85.0000]])
```

연산	모양	뜻
*	같은 모양 (또는 브로드캐스팅)	같은 자리끼리 곱
@	(m, k) @ (k, n) → (m, n)	곱해서 더하기
점수 @ 비율	(3, 2) @ (2,) → (3,)	학생별 가중 평균
점수 @ 두_기준	(3, 2) @ (2, 2) → (3, 2)	기준 두 개를 한 번에

신경망과 연결

nn.Linear = 입력 @ 가중치.T + 편향
6장 모델의 핵심 계산이 바로 @

b18 축 접기: max·argmax와 keepdim

```
print(점수.sum(), 점수.mean())      # 전부 접기
print(점수.mean(dim=0))              # 학생 축이 사라짐 → 과목별
print(점수.mean(dim=1))              # 과목 축이 사라짐 → 학생별
print(점수.mean(dim=1, keepdim=True).shape)
최고 = 점수.max(dim=0)               # 값과 위치를 함께
print(최고.values, 최고.indices)
print(점수.argmax(dim=1))            # 위치만
```

```
tensor(500.) tensor(83.3333)
tensor([82.3333, 84.3333])
tensor([75., 90., 85.])
torch.Size([3, 1])
tensor([92., 95.]) tensor([1, 2])
tensor([0, 0, 1])
```

함수	돌려주는 것	dim=0 예
max	값 + 위치	[92, 95], [1, 2]
argmax	위치만	[1, 2]
min / argmin	가장 작은 값 쪽	[75, 70], [2, 0]

max(dim=0) 읽기

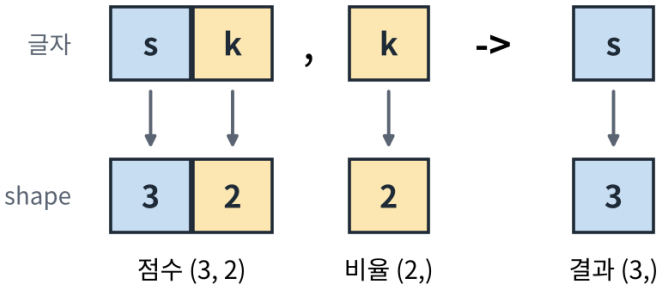
과목마다 가장 높은 점수와 그 학생 번호
국어 92점은 1번, 수학 95점은 2번 학생

keepdim=True

접은 축을 크기 1로 남김 → (3, 1)
원래 (3, 2)와 바로 브로드캐스팅 가능 (b23 표준화)

einsum 읽는 법: 글자는 내가 붙이는 이름표

torch.einsum("sk,k->s", 점수, 비율): 글자를 shape 자리에 순서대로 붙임



s = 3 (학생 축), k = 2 (과목 축)

토치는 글자의 '뜻'은 모름. 몇 번째 자리인지만 봄

① 글자 순서 = 축 순서

첫 글자 → 0번 축, 둘째 글자 → 1번 축

② 같은 글자 = 같은 축

점수의 k(2)와 비율의 k(2): 크기가 같아야 함

③ -> 뒤에 없는 글자는 사라짐

k가 없음 → k 방향으로 곱해서 더함 (가중합)

④ -> 뒤 글자 순서 = 결과 모양

"s" → (3,) "sk->ks" → 전치 (2, 3)

"ab,b->a"도 같은 결과 [74, 89.6, 87]

"sk,s->k" → 오류: s가 점수에선 3, 비율에선 2

```
torch.einsum("sk,k->s", 점수, 비율)
torch.einsum("ab,b->a", 점수, 비율)   # 글자만 바꿔도 같음
torch.einsum("sk,s->k", 점수, 비율)   # s: 3 vs 2 → 오류
```

```
tensor([74.0000, 89.6000, 87.0000])
tensor([74.0000, 89.6000, 87.0000])
오류: subscript s has size 2 for operand 1 ...
```

b30 einsum: 축 이름으로 계산 적기

```
점수 = torch.tensor([[80.0, 70.0],
                     [92.0, 88.0],
                     [75.0, 95.0]])   # (학생 s, 과목 k)
비율 = torch.tensor([0.4, 0.6])       # (과목 k)
print(torch.einsum("sk,k->s", 점수, 비율))
print(torch.einsum("sk->ks", 점수))   # 전치
print(torch.einsum("sk->k", 점수))    # s가 사라짐

반별점수 = torch.stack([점수, 점수 - 10])    # (b, s, k) = (2, 3, 2)
기준 = torch.tensor([[[0.4, 0.5], [0.6, 0.5]],
                     [[0.5, 0.3], [0.5, 0.7]]])  # (b, k, c)
결과 = torch.einsum("bsk,bkc->bsc", 반별점수, 기준)
print(결과.shape, torch.allclose(결과, 반별점수 @ 기준))
```

```
tensor([74.0000, 89.6000, 87.0000])
tensor([[80., 92., 75.],
        [70., 88., 95.]])
tensor([247., 253.])
torch.Size([2, 3, 2]) True
```

einsum 식	같은 계산
"sk,k->s"	점수 @ 비율
"sk->ks"	점수.T
"sk->k"	점수.sum(dim=0)
"bsk,bkc->bsc"	반별점수 @ 기준

어디서 보나

@도 앞 차원을 배치로 보고 반마다 따로 곱함
→ allclose True
트랜스포머 어텐션 코드에서 "bqd,bkd->bqk" 같은 식을 자주 봄

b31 sort, argsort, topk: 순위 매기기

```
점수 = torch.tensor([72.0, 95.0, 88.0, 60.0, 91.0, 79.0])
정렬 = 점수.sort(descending=True)
print(정렬.values)
print(정렬.indices)           # 원래 몇 번째였는지
print(점수.argsort())         # 오름차순 번호만
상위 = 점수.topk(3)           # 큰 값 3개만
print(상위.values, 상위.indices)

확률 = torch.tensor([[0.05, 0.60, 0.10, 0.25],
                     [0.40, 0.05, 0.35, 0.20]])
값, 번호 = 확률.topk(2, dim=1)   # 결과 두 개를 나눠 담음
정답 = torch.tensor([3, 1])
print((번호 == 정답.unsqueeze(1)).any(dim=1))
```

```
tensor([95., 91., 88., 79., 72., 60.])
tensor([1, 4, 2, 5, 0, 3])
tensor([3, 0, 5, 2, 4, 1])
tensor([95., 91., 88.]) tensor([1, 4, 2])
번호 [[1, 3], [0, 2]]  값 [[0.60, 0.25], [0.40, 0.35]]
상위 2개 안에 정답이 있나: tensor([ True, False])
```

함수	돌려주는 것
sort	정렬된 값 + 원래 번호
argsort	번호만 (기본 오름차순)
topk(k)	큰 값 k개 + 번호 (전체 정렬보다 빠름)

top-k 정확도

정답이 상위 k개 안에 있으면 맞힌 것으로 침
번호 (2, 2) == 정답 (2, 1) → 브로드캐스팅 비교 → any(dim=1)
1번 사진 정답 1은 확률 0.05라 빠짐

확인 문제: 한꺼번에 계산하기

15 국어 >= 수학과 torch.maximum(국어, 수학)의 결과는?

```
torch.maximum(국어, 수학)
```

16 점수 + torch.tensor([[1.0], [2.0]])은?

```
(3, 2) + (2, 1)
```

17 비율을 [0.5, 0.5]로 바꾸면?

```
비율 = torch.tensor([0.5, 0.5])
```

18 점수.max(dim=1)의 values와 indices는?

```
최고 = 점수.max(dim=1)
```

19 a += 10과 a = a + 10, 어느 쪽이 같은 텐서를 바꿀까?

```
print(id(a)) 앞뒤 비교
```

30 einsum("sk->s", 점수)는 무엇과 같을까?

```
torch.einsum("sk->s", 점수)
```

31 점수.topk(2, largest=False)의 결과는?

```
점수.topk(2, largest=False)
```

확인 문제 정답: 한꺼번에 계산하기

15 [True, True, False]
[80, 92, 95] 칸마다 큰 값

16 오류: The size of tensor a (3) must match the size of tensor b (2) at non-singleton dimension 0

17 [75., 90., 85.] 단순 평균 = 점수.mean(dim=1)

18 values [80, 92, 95], indices [0, 0, 1]: 학생별 최고점과 그 과목

19 id(): 객체마다 붙는 고유 번호
a += 10은 제자리 (id 그대로, add_와 같음)
a = a + 10은 새 텐서를 만들어 이름만 옮김

30 [150, 180, 170] = 점수.sum(dim=1)
k가 사라짐

31 values [60, 72], indices [3, 0]: 가장 낮은 두 명

6 장치 · 저장 · 주의점

b20~b22 GPU · b32 b33 저장 · b34 NumPy와 차이 · b35 자주 만나는 오류

b20 장치(device): CPU와 GPU 오가기

```
def 장치_고르기() -> torch.device:   # 장치를 돌려주는 함수
    if torch.cuda.is_available():
        return torch.device("cuda")     # NVIDIA GPU
    if torch.backends.mps.is_available():
        return torch.device("mps")      # Apple 실리콘
    return torch.device("cpu")

장치 = 장치_고르기()
print("고른 장치:", 장치)
x = torch.ones(3)                    # 처음엔 CPU에
print(x.device)
y = x.to(장치)                       # 새 텐서를 돌려줌
print(y.device)
z = torch.ones(3, device=장치)       # 처음부터 그 장치에
print((y + z).cpu())                 # 결과를 CPU로
```

```
고른 장치: cpu
cpu
cpu
tensor([2., 2., 2.])
```

장치	언제
cuda	NVIDIA GPU (Colab GPU 런타임)
mps	Apple 실리콘 Mac (M1~)
cpu	기본값, 어디서나

모델은 model.to(장치)로 통째로 옮김 (6장)

기억할 것

x.to(장치)는 새 텐서 → x = x.to(장치)로 다시 담기
함께 계산하는 텐서는 모두 같은 장치
출력·NumPy 변환은 .cpu() 뒤에

b21·b22 장치가 다르면 오류, GPU 시간은 기다렸다 재기

```
if torch.backends.mps.is_available():
    gpu = torch.ones(3, device="mps")
    cpu = torch.ones(3)
    try:
        gpu + cpu                # 장치가 다름
    except RuntimeError as e:
        print("오류:", str(e)[:120])
    print(gpu + cpu.to("mps"))  # 맞추면 OK
```

b21: 장치가 섞이면 RuntimeError

GPU 텐서 + CPU 텐서는 계산 불가 → 한쪽을 .to()로 옮겨 맞춤
CPU만 있는 환경에선 이 오류를 못 보고 넘어가기 쉬움 (확인 문제)

```
a = torch.randn(4096, 4096); b = torch.randn(4096, 4096)
시작 = time.perf_counter(); a @ b    # 스톱워치 (b01)
print(f"CPU: {time.perf_counter() - 시작:.3f}초")
if torch.backends.mps.is_available():
    ga, gb = a.to("mps"), b.to("mps")
    ga @ gb                      # 준비 운동
    torch.mps.synchronize()
    시작 = time.perf_counter(); ga @ gb
    torch.mps.synchronize()      # 끝날 때까지 기다림
    print(f"MPS: {time.perf_counter() - 시작:.3f}초")
```

```
CPU: 0.643초    (실행 결과 예, 컴퓨터마다 다름)
```

GPU 시간을 잴 때 두 가지

① 첫 실행은 준비 시간이 섞임 → 한 번 돌린 뒤 잼
② GPU는 일을 맡기고 바로 돌아옴(비동기) → synchronize()로 끝날 때까지 기다린 뒤 시간 측정 (CUDA는 torch.cuda.synchronize())

b32·b33 텐서 저장과 불러오기

```
# with 임시 폴더: 블록이 끝나면 폴더가 자동으로 지워짐
# 경로 = 폴더 / "기온.pt"  (Path는 / 로 경로를 이어 붙임)
torch.save(기온, 경로)
다시 = torch.load(경로)
print(다시.shape, torch.equal(기온, 다시))

torch.save({"기온": 기온, "도시": ["가람시", "나래시", "다솜시"],
            "단위": "°C"}, 폴더 / "묶음.pt")
묶음 = torch.load(폴더 / "묶음.pt")
print(묶음["도시"], 묶음["기온"][0])

큰_텐서 = torch.zeros(1_000_000)
torch.save(큰_텐서[:10], 폴더 / "조각.pt")
torch.save(큰_텐서[:10].clone(), 폴더 / "복사.pt")
```

```
torch.Size([3, 4]) True
['가람시', '나래시', '다솜시'] tensor([ 2.0000, 13.5000, 25.7000, 15.0000])
조각.pt 4,001,577바이트
복사.pt 1,577바이트
```

숫자 10개인데 4MB?

슬라이스는 원본 저장공간(100만 칸 × 4바이트)을 공유(b11) → 통째로 저장됨
잘라서 저장할 땐 .clone()

```
x = torch.arange(3.0, device=장치_고르기())
torch.save(x, 경로)
print(torch.load(경로).device)
print(torch.load(경로,
      map_location="cpu").device)
```

```
cpu
cpu       (CPU 환경 실행)
```

b33 map_location

텐서는 저장할 때의 장치로 돌아옴
GPU에서 저장한 파일을 GPU 없는 컴퓨터에서 열면 오류 → map_location="cpu"로 불러오면서 옮김

b34 NumPy와 다른 점

```
a = np.array([[1.0, 2.0], [3.0, 4.0]])
t = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
print(a.sum(axis=0), t.sum(dim=0))
print(np.concatenate([a, a]).shape, torch.cat([t, t]).shape)
print(a.astype(np.int64).dtype, t.to(torch.int64).dtype)
print(a.max(axis=1))
print(t.max(dim=1))      # 값과 위치를 함께
값 = [2.0, 4.0, 4.0, 6.0]
print(np.std(값), torch.tensor(값).std())
print(np.std(값, ddof=1), torch.tensor(값).std(correction=0))
```

```
[4. 6.] tensor([4., 6.])
(4, 2) torch.Size([4, 2])
int64 torch.int64
[2. 4.]
torch.return_types.max(values=tensor([2., 4.]), indices=tensor([1, 1]))
1.4142135623730951 tensor(1.6330)
1.632993161855452 tensor(1.4142)
```

NumPy	PyTorch
axis=0	dim=0
np.concatenate	torch.cat
astype(np.int64)	to(torch.int64)
max(axis=1): 값만	max(dim=1): 값 + 위치
std: n으로 나눔	**std: n − 1로 나눔**

표준편차가 다르면

버그가 아니라 기본값 차이
맞추려면 np.std(ddof=1) 또는
torch.std(correction=0)

손계산: 평균 4, 편차² 합 = 4 + 0 + 0 + 4 = 8 → 8 ÷ 4 = 2 → √2 = 1.414 (NumPy) · 8 ÷ 3 → 1.633 (PyTorch)

b35 자주 만나는 오류 네 가지

```
# 실제 파일은 오류_보기(설명, lambda: 계산)로 감싸서
# 오류가 나도 멈추지 않고 메시지 첫 줄만 출력
x = torch.randn(4, 3)          # 사진 4장 × 특성 3개
w = torch.randn(4, 2)          # 실수: (3, 2)여야 함
x @ w
torch.tensor([[1, 2]]) @ torch.tensor([[0.5], [0.5]])
x.sum(dim=0).item()
torch.stack([torch.zeros(3), torch.zeros(4)])

w = torch.randn(3, 2)          # 고친 뒤
print((x @ w).shape)
print((torch.tensor([[1, 2]]).float() @ ...).item())
print(x.sum().item())
```

```
[행렬 곱 모양] mat1 and mat2 shapes cannot be
  multiplied (4x3 and 4x2)
[자료형 섞기] expected m1 and m2 to have the
  same dtype, but got: long int != float
[숫자 하나가 아님] a Tensor with 3 elements
  cannot be converted to Scalar
[쌓을 크기가 다름] stack expects each tensor to
  be equal size, but got [3] at entry 0 and [4] ...
(4, 3) @ (3, 2) → torch.Size([4, 2])
1.5
-3.141735553741455
```

메시지 핵심	원인	고치는 법
shapes cannot be multiplied (4x3 and 4x2)	@ 안쪽 크기 3 ≠ 4	w를 (3, 2)로, 또는 .T
same dtype … long int != float	정수 × 실수 행렬 곱	.float()로 맞추기
3 elements cannot be converted to Scalar	.item()은 숫자 1개만	sum()까지 접거나 .tolist()
stack expects each tensor to be equal size	쌓을 텐서 크기가 다름	크기 맞추기, 또는 cat

디버깅 습관: 오류가 나면 관련 텐서의 .shape, .dtype, .device부터 print

확인 문제: 장치·저장·주의점

20 CUDA GPU에서 z를 장치 없이 torch.ones(3)으로 만들면 y + z는?

```
z = torch.ones(3)
```

21 이 코드를 Colab(NVIDIA GPU)에서 확인하려면 어디를 바꿀까?

```
torch.backends.mps.is_available()
```

22 synchronize() 두 줄을 지우면 GPU 시간은?

```
# torch.mps.synchronize()
```

32 큰_텐서[:10]을 작은 파일로 저장하려면?

```
torch.save(큰_텐서[:10]???, 경로)
```

33 GPU에서 저장한 x.pt를 GPU 없는 노트북에서 그냥 load하면?

```
torch.load(경로)
```

34 PyTorch로 np.std(값)과 같은 1.4142를 얻으려면?

```
torch.tensor(값).std(?)
```

35 첫 번째 오류가 사라지게 w를 고치려면?

```
w = torch.randn(?, ?)
```

확인 문제 정답: 장치·저장·주의점

20 RuntimeError: 장치가 다름 (cuda:0과 cpu)
CPU만 있으면 둘 다 cpu라 오류가 안 나서 놓치기 쉬움

21 torch.cuda.is_available()로
"mps"는 모두 "cuda"로 바꿈

22 실제보다 훨씬 짧게 나옴
GPU 계산이 끝나기 전에 시간을 재 버림 (비동기)
CUDA는 torch.cuda.synchronize()

32 큰_텐서[:10].clone() 저장 → 1,577바이트
슬라이스는 원본 저장공간 전체(4MB)를 끌고 감

33 오류 (GPU 장치를 찾을 수 없음)
torch.load(경로, map_location="cpu")로 불러옴

34 std(correction=0)
PyTorch 기본은 n − 1로 나누는 표본 표준편차

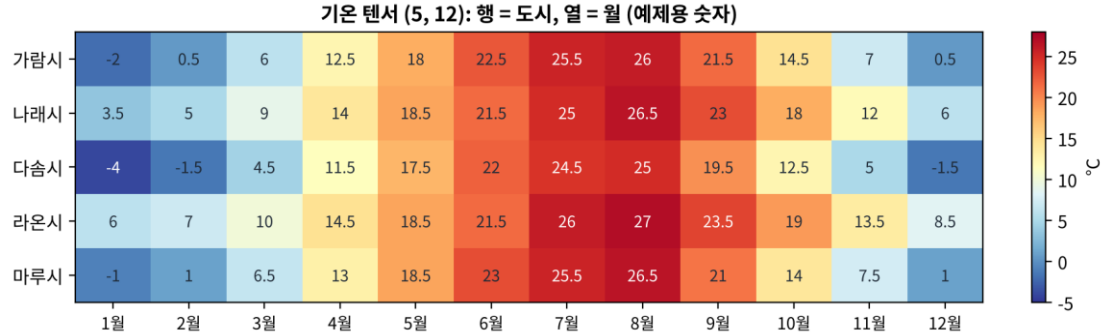
35 w = torch.randn(3, 2) → (4, 3) @ (3, 2) = (4, 2)
안쪽 3끼리 맞아야 함

7

실습: 5개 도시 기온 분석

b23 · 축 접기 · 브로드캐스팅 · 행렬 곱 · topk · where를 한 데이터에

b23 ① 데이터와 도시별 요약



도시	연평균	연교차	더운 달
가람시	12.71	28.0	8월
나래시	15.17	23.0	8월
다솜시	11.25	**29.0**	8월
라온시	**16.25**	21.0	8월
마루시	13.04	27.5	8월

```
print("모양:", 기온.shape)                  # (5, 12)
연평균 = 기온.mean(dim=1)
연교차 = 기온.max(dim=1).values - 기온.min(dim=1).values
print("가장 더운 달:", (기온.argmax(dim=1) + 1).tolist())  # 0부터라 +1
```

모양 따라가기

기온 (5, 12)
.mean(dim=1) → (5,)
.max(dim=1).values → (5,)
argmax(dim=1) → (5,) 0~11

읽어 보기

라온시가 가장 따뜻하고 연교차가 작음
다솜시는 겨울이 추워 연교차가 가장 큼
모든 계산이 반복문 없이 dim 하나로

b23 ② 표준화하고 닮은 도시 찾기

```
평균 = 기온.mean(dim=1, keepdim=True)          # (5, 1)
표준편차 = 기온.std(dim=1, keepdim=True)       # (5, 1)
표준화 = (기온 - 평균) / 표준편차              # (5, 12)
print(표준화.mean(dim=1).abs().max() < 1e-6)

단위 = 표준화 / 표준화.norm(dim=1, keepdim=True)
유사도 = 단위 @ 단위.T                         # (5, 5)
유사도.fill_diagonal_(-1.0)                    # 자기 자신 빼기
짝 = 유사도.argmax(dim=1)
```

```
표준화 후 도시별 평균: tensor(True)
가람시와 가장 닮은 도시: 마루시 (유사도 0.9992)
나래시와 가장 닮은 도시: 라온시 (유사도 0.9974)
다솜시와 가장 닮은 도시: 마루시 (유사도 0.9996)
라온시와 가장 닮은 도시: 나래시 (유사도 0.9974)
마루시와 가장 닮은 도시: 다솜시 (유사도 0.9996)
```

fill_diagonal_은 제자리 연산

대각선(자기 자신과의 유사도 1)을 −1로 덮어써서 argmax에서 빠지게 함 (b19 밑줄)

단계	모양
평균, 표준편차 (keepdim)	(5, 1)
기온 − 평균 브로드캐스팅	(5, 12)
÷ norm → 길이 1	(5, 12)
단위 @ 단위.T	(5, 5)
fill_diagonal_(−1)	자기 자신 제외
argmax(dim=1)	(5,) 짝 번호

코사인 유사도

길이 1로 맞춘 두 줄을 곱해서 더하면 −1~1
표준화로 '얼마나 더운지'는 빼고 '오르내리는 모양'만 비교 → 모두 0.99 이상

b23 ③ 가장 더운 석 달과 난방 지수

```
더운_석달 = 기온.topk(3, dim=1).indices.sort(dim=1).values + 1

난방 = torch.where(기온 < 10, 10 - 기온, 0.0)
print("난방이 필요한 달 수:", (기온 < 10).sum(dim=1).tolist())
print("난방 지수:", 난방.sum(dim=1).tolist())
print("clamp로 계산해도 같은가:",
      torch.equal(난방, (10 - 기온).clamp(min=0)))
```

```
가람시 [6, 7, 8]월  나래시 [7, 8, 9]월  다솜시 [6, 7, 8]월
라온시 [7, 8, 9]월  마루시 [6, 7, 8]월
난방이 필요한 달 수: [5, 4, 5, 3, 5]
난방 지수: [38.0, 16.5, 47.5, 8.5, 35.0]
clamp로 계산해도 같은가: True
```

난방 지수 = 10도에 모자란 만큼의 합

10도 미만인 달만 (10 − 기온)을 더함
True는 1로 세어서 (기온 < 10).sum(dim=1) = 달 수

도시	10도 미만 달	난방 지수
가람시	5	38.0
나래시	4	16.5
다솜시	5	**47.5**
라온시	3	8.5
마루시	5	35.0

왜 sort를 한 번 더?

topk 번호는 '큰 순서' [7, 6, 8] 식 → 달 순서로 보려고 정렬

확인 문제: 실습

23 도시마다 가장 추운 달과 그 기온은?

```
기온.min(dim=1)
```

23 영하인 달은 도시마다 몇 개?

```
(기온 < 0).sum(dim=1)
```

23 연교차가 가장 큰 도시는?

```
도시[연교차.argmax()]
```

23 난방 기준을 15도로 올리면 난방 지수와 달 수는?

```
torch.where(기온 < 15, 15 - 기온, 0.0)
```

23 월별로 다섯 도시의 평균 기온은? 모양은?

```
기온.mean(dim=0)
```

23 유사도 표에서 fill_diagonal_(-1.0) 줄을 지우면 짝은?

```
# 유사도.fill_diagonal_(-1.0)
```

확인 문제 정답: 실습

23 모두 1월 (indices 0).
값 [−2.0, 3.5, −4.0, 6.0, −1.0]

23 [1, 0, 3, 0, 1]
True를 1로 세어 더함

23 다솜시 (29.0°C)

23 지수 [66.0, 40.5, 78.5, 30.5, 63.0]
달 수 [7, 6, 7, 6, 7]

23 (12,)
[0.5, 2.4, 7.2, 13.1, 18.2, 22.1,
25.3, 26.2, 21.7, 15.6, 9.0, 2.9]
dim=0 → 도시 축이 사라짐

23 모든 도시가 자기 자신(유사도 1.0)을 고름
→ [0, 1, 2, 3, 4]

정리

01	만들기	tensor(목록) · zeros/ones/arange · 씨앗으로 재현 · from_numpy는 공유
02	모양·자료형	shape는 바깥→안쪽, 이미지는 (N, C, H, W). 기본 float32, 메모리 = 바이트 × 개수
03	인덱싱	[행, 열] · 마스크 · 슬라이스는 공유(clone) · where/clamp · gather/scatter
04	모양 바꾸기	reshape(-1) · permute · unsqueeze/squeeze · cat vs stack · stride와 contiguous
05	계산	원소별 · 브로드캐스팅(뒤에서부터 같거나 1) · @ · dim=k면 k축이 사라짐
06	장치·저장	같은 장치끼리 계산 · map_location · 오류는 shape/dtype/device부터 확인